Senior DevOps Portfolio Project

Internal Developer Platform (IDP) — Build Specification

Objective

Build a self-service Internal Developer Platform: a Kubernetes-based platform other engineers could deploy applications onto, provisioned entirely through code, observed end-to-end, and secured by policy. The goal is to demonstrate platform-level ownership — not "I deployed an app," but "I built the golden path other engineers deploy on." This is the framing that reads as senior/staff-level to interviewers.

What to Build

1. Infrastructure as Code (foundation layer)

Provision all cloud infrastructure through Terraform: VPC/networking, a managed Kubernetes cluster, IAM roles, and remote state with locking. Structure it as reusable modules across dev/stage/prod environments — this is the first thing a senior reviewer checks for.

Tools: Terraform, AWS (VPC, EKS, IAM, S3+DynamoDB for state)

2. Container Orchestration

Stand up the Kubernetes cluster that hosts everything else. Optionally run OKD (the free, upstream project OpenShift is built on) alongside or instead of EKS to close the OpenShift gap directly.

Tools: Amazon EKS, OKD (OpenShift upstream), kubectl, Helm

3. GitOps Delivery

All deployments should be declarative and pulled from Git, not pushed manually. This is the pattern senior DevOps/platform roles expect by default.

Tools: ArgoCD, Kustomize

4. CI/CD Pipelines

Build pipelines that lint, test, scan, and build container images before ArgoCD picks up the change. Security scanning in the pipeline (not bolted on after) is a senior-level signal.

Tools: GitHub Actions, Trivy (image scanning), a SAST tool (e.g. Semgrep)

5. Observability Stack

Metrics, logs, and dashboards for every service running on the platform. Standing up ELK yourself (rather than a managed logging add-on) directly closes the ELK/Splunk gap flagged in your resume review.

Tools: Prometheus, Grafana, Elasticsearch, Logstash, Kibana (ELK)

6. Security & Policy as Code

Enforce guardrails automatically rather than through manual review: no privileged containers, no unscanned images, secrets never in plaintext.

Tools: OPA/Gatekeeper or Kyverno, HashiCorp Vault, Kubernetes RBAC

7. Self-Service Developer Portal

A thin UI where a developer can request a new service, see its deployment status, and find its docs — this is the piece that turns "a cluster" into "a platform."

Tools: Backstage

8. SRE Practices

Define SLOs/SLIs for the platform itself, wire up alerting against them, and write runbooks. Optionally run a controlled failure injection test.

Tools: Prometheus Alertmanager, Grafana SLO dashboards, Litmus (chaos testing, optional)

Optional Stretch Module: Cost & Data Pipeline

A small dbt + SQL pipeline that transforms platform cost or log data into a cost-visibility dashboard. This is the one piece that lets you touch the SQL/dbt gap without it feeling bolted on to a DevOps project.

- dbt
- SQL (Postgres or a warehouse like BigQuery/Snowflake free tier)
- Grafana or a BI tool for the dashboard

How This Maps to Your Flagged Skill Gaps

Gap flagged in resume review	Addressed by
OpenShift 4	OKD cluster (component 2)
ELK / Splunk	Self-built ELK stack (component 5)
SQL / dbt	Optional cost/data pipeline module
Dynatrace	Not directly covered — Prometheus/Grafana is the open-source equivalent; mention this substitution explicitly in interviews

Suggested Build Order

- Terraform foundation (VPC, EKS/OKD cluster, remote state)
- GitOps + CI/CD wiring (ArgoCD + GitHub Actions with scanning)
- Observability (Prometheus/Grafana, then ELK)

- Security and policy as code (OPA/Kyverno, Vault, RBAC)
- Backstage developer portal
- SRE layer (SLOs, alerting, runbooks, optional chaos test)
- Optional: dbt/SQL cost pipeline

Prepared as a build reference for a Senior DevOps Engineer portfolio project.